

Numerical on calculating the weight for a simple perceptron

Numerical 1: We want to train a perceptron to implement the following Boolean function:

Example No	x1	x2	x3	y (target output)
1	0	0	0	0
2	0	0	1	1
3	0	1	0	1
4	0	1	1	1
5	1	0	0	1
6	1	0	1	1
7	1	1	0	1
8	1	1	1	1

Initialization:

- Weights:** Initialize the weights (w_1, w_2, w_3) and bias (b) to small random numbers or zero. Let's start with:
 - $w_1 = 0.1$
 - $w_2 = -0.2$
 - $w_3 = 0.3$
 - $b = 0$
- Learning Rate (α or η):** Choose a learning rate. This controls how much the weights are adjusted in each iteration. Let $\alpha = 0.1$.

Training:

We'll go through the training data one example at a time (this is called "stochastic" or "online" learning).

Epoch 1:

- Example 1: $x_1 = 0, x_2 = 0, x_3 = 0, y = 0$**
 - Calculate the weighted sum:** $(0.1 * 0) + (-0.2 * 0) + (0.3 * 0) + 0 = 0$
 - Apply the activation function:** Let's use a simple step function (1 if sum ≥ 0 , 0 otherwise). Since $0 \geq 0$, the output is 1.
 - Calculate the error:** $error = target - output = 0 - 1 = -1$
 - Update the weights and bias:**

Weight update rule:

$$w_i = w_i + \alpha * error * x_i$$

$$b = b + \alpha * error$$

- $w_1 = w_1 + \alpha * error * x_1 = 0.1 + 0.1 * (-1) * 0 = 0.1$
- $w_2 = w_2 + \alpha * error * x_2 = -0.2 + 0.1 * (-1) * 0 = -0.2$

- $w_3 = w_3 + \alpha * \text{error} * x_3 = 0.3 + 0.1 * (-1) * 0 = 0.3$
- $b = b + \alpha * \text{error} = 0 + 0.1 * (-1) = -0.1$

- **Example 2: $x_1 = 0, x_2 = 0, x_3 = 1, y = 1$**
 1. **Calculate the weighted sum:** $(0.1 * 0) + (-0.2 * 0) + (0.3 * 1) + (-0.1) = 0.2$
 2. **Apply the activation function:** 1 (since $0.2 \geq 0$)
 3. **Calculate the error:** $1 - 1 = 0$
 4. **Update the weights and bias:** No updates because the error is 0.
- **Continue for all other examples...**

Epoch 2 (and subsequent epochs):

Repeat the process for all training examples. You might need to go through the training data multiple times (epochs) until the perceptron converges (i.e., it correctly classifies all examples) or until you reach a maximum number of epochs.

Important Notes: (extra)

- **Convergence:** perceptrons can converge to a correct solution if the data is linearly separable (i.e., you can draw a line or hyperplane to separate the classes).
 - **Non-linearly Separable Data:** If the data is not linearly separable, the perceptron might not converge. More complex neural networks are needed for such cases.
 - **Activation Function:** The choice of activation function is important. We used a simple step function here. Other common options include the sigmoid function.
 - **Bias:** The bias term (b) is like an extra weight connected to a constant input of 1. It allows the decision boundary to shift.
-

Numerical 2:

Training Dataset (Input & Target Output)

X1	X2	Target (y)
0	0	0
0	1	1

Initial Conditions

- **Initial Weights:** $w_1 = 0.1, w_2 = -0.2$
- **Initial Bias:** $b = 0.05$
- **Learning Rate:** η or $\alpha = 0.1$
- **Activation Function (Step Function)**

$$y_{pred} = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- **Weight Update Rule (Perceptron Learning Rule)**

$$w_i = w_i + \eta \cdot (y - y_{pred}) \cdot x_i$$

$$b = b + \eta \cdot (y - y_{pred})$$

Epoch 1: Iterating Over Each Training Example

Step 1: Compute Weighted Sum (for each row in dataset)

Net Input (Weighted Sum Calculation)

$$z = (w_1 \cdot x_1) + (w_2 \cdot x_2) + b$$

X1	X2	Net Input (z)	y_pred (Step Activation)	Error (y - y_pred)	Weight Update
0	0	$(0.1 \times 0) + (-0.2 \times 0) + 0.05 = 0.05$	1	$0 - 1 = -1$	$w_1 = 0.1, w_2 = -0.2, b = 0.05 - 0.1 = -0.05$
0	1	$(0.1 \times 0) + (-0.2 \times 1) - 0.05 = -0.25$	0	$1 - 0 = 1$	$w_2 = -0.2 + (0.1 \times 1) = -0.1, b = -0.05 + 0.1 = 0.05$

Final Updated Weights and Bias

After the first epoch, we get:

$$w_1 = 0.1, w_2 = -0.1, b = 0.05$$

Since the perceptron now **correctly classifies all training examples**, additional epochs will **not change the weights further**.

Conclusion

The perceptron **converged in one epoch** as the dataset is linearly separable.
The final learned decision boundary is defined by:

$$x_1 \cdot 0.1 + x_2 \cdot (-0.1) + 0.05 = 0$$

Numerical 3:

Training the Perceptron for OR Gate

Step 1: Initialize Weights and Bias

Let's assume:

- $w_1 = 0.2, w_2 = 0.3, b = 0.1$
- Learning rate $\eta = 0.1$
- Number of **epochs**: any integer number say 10 (Iterations over the dataset)

Step 2: Compute Net Input & Predictions

We apply the **Perceptron Algorithm** to adjust weights.

X1	X2	Net Input z	Predicted y_{pred}	Actual y	Error $e =$ $y - y_{\text{pred}}$	Weight Update
0	0	$(0.2 \times 0) + (0.3 \times 0) + 0.1 = 0.1$	1	0	-1	$w_1 = 0.2 + (0.1 \times -1 \times 0) = 0.2, w_2 = 0.3, b = 0.1 - 0.1 = 0.0$
0	1	$(0.2 \times 0) + (0.3 \times 1) + 0.0 = 0.3$	1	1	0	No Update
1	0	$(0.2 \times 1) + (0.3 \times 0) + 0.0 = 0.2$	1	1	0	No Update
1	1	$(0.2 \times 1) + (0.3 \times 1) + 0.0 = 0.5$	1	1	0	No Update

Updated Weights after Iteration 1:

$$w_1 = 0.2, w_2 = 0.3, b = 0.0$$

Since there are **no more errors**, the perceptron has learned the OR function.

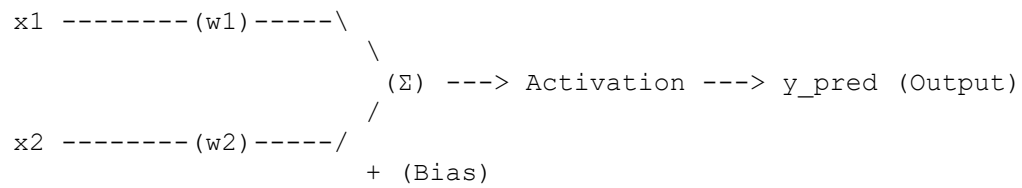
OR Iteration 2 (Check for Errors)

- Compute net input and predictions again.
- If no more errors, training **stops**.

Step 3: Final Weights

After a few iterations, the weights **converge**, and the perceptron correctly classifies all OR function inputs.

Diagram of the Perceptron for OR Gate



Mathematically,

$$y_{pred} = f(w_1x_1 + w_2x_2 + b)$$

where $f(z)$ is the **Step Activation Function**.

Decision Boundary for OR Function

Once we have the trained weights, we can derive the **decision boundary equation**:

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

By plotting this line, we visualize the separation of **class 0 and class 1**.

Numerical on Activation Function

Study to obtain the value of neuron Y for a network using activation functions

To obtain the value of a neuron Y in a neural network using the sigmoid activation function, we need to go through a few steps:

Steps to Calculate Neuron Y

1. Compute the Weighted Sum of Inputs:

- For a neuron Y , you typically have several input values $X_1, X_2, \dots, X_n$, each associated with a weight $w_1, w_2, \dots, w_n$.
- The weighted sum z is calculated as:

$$z = w_1 \cdot X_1 + w_2 \cdot X_2 + \dots + w_n \cdot X_n + b$$

- Here, b is the bias term associated with the neuron.

2. Apply the Sigmoid Activation Function:

- The sigmoid function $\sigma(z)$ is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- This function maps the weighted sum z to a value between 0 and 1.

3. Obtain the Output:

- The output of the neuron Y is the result of applying the sigmoid function to the weighted sum z .

Example Calculation

Suppose we have the following values:

- Inputs: $X_1 = 0.5, X_2 = 0.3$
- Weights: $w_1 = 0.8, w_2 = -0.5$
- Bias: $b = 0.1$

Step 1: Compute the Weighted Sum z :

$$z = (0.8 \times 0.5) + (-0.5 \times 0.3) + 0.1$$
$$z = 0.4 - 0.15 + 0.1 = 0.35$$

Step 2: Apply the Sigmoid Activation Function:

$$\sigma(0.35) = \frac{1}{1 + e^{-0.35}}$$

Let's calculate this:

$$e^{-0.35} \approx 0.704$$
$$\sigma(0.35) = \frac{1}{1 + 0.704} \approx \frac{1}{1.704} \approx 0.586$$

Step 3: Output of Neuron Y : The output value of the neuron Y is approximately 0.586.

Numerical 2

Obtain the value of a neuron Y in a neural network using the sigmoid activation function.

Suppose we have the following values:

- Inputs: $x_1 = 0.7, x_2 = 0.3, x_3 = 0.6, x_4 = 0.5$
- Weights: $w_1 = 0.1, w_2 = -0.2, w_3 = 0.4, w_4 = 0.3$
- Bias: $b = 0.45$

Step 1: Compute the Weighted Sum z :

$$z = (0.7 \times 0.1) + (0.3 \times -0.2) + (0.6 \times 0.4) + (0.5 \times 0.3) + 0.45 = 0.85$$

Step 2: Apply Sigmoid Activation Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.85}} = 0.7005$$

Step 3: Output of Neuron Y : The output value of the neuron Y is approximately 0.7005

To calculate the output of a neuron Y in a neural network using the **sigmoid activation function** and the **tanh activation function**, we need to follow these steps:

1. Compute the Weighted Sum (Linear Combination) Z:

Given the inputs $x_1, x_2, \dots, x_n$, their corresponding weights $w_1, w_2, \dots, w_n$, and the bias b , the weighted sum z is calculated as:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

2. Apply the Sigmoid Activation Function:

The sigmoid activation function, $\sigma(z)$, is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

3. Apply the Tanh Activation Function:

The tanh activation function, $\tanh(z)$, is defined as:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Example Calculation: **Numerical 1**

Let's assume the following values for the inputs, weights, and bias:

- Inputs: $x_1 = 0.5, x_2 = -1.5, x_3 = 2.0$
- Weights: $w_1 = 0.4, w_2 = -0.7, w_3 = 0.2$
- Bias: $b = 0.1$

Step 1: Calculate the weighted sum z

$$z = (0.4 \times 0.5) + (-0.7 \times -1.5) + (0.2 \times 2.0) + 0.1$$

$$z = 0.2 + 1.05 + 0.4 + 0.1 = 1.75$$

Step 2: Apply the Sigmoid Activation Function

$$Y_{\text{sigmoid}} = \sigma(z) = \frac{1}{1 + e^{-1.75}}$$

Let's calculate this value:

$$Y_{\text{sigmoid}} = \frac{1}{1 + e^{-1.75}} \approx \frac{1}{\downarrow 0.17377} \approx \frac{1}{1.17377} \approx 0.851$$

So, the output Y_{sigmoid} using the sigmoid activation function is approximately **0.851**.

Step 3: Apply the Tanh Activation Function

$$Y_{\text{tanh}} = \tanh(z) = \frac{e^{1.75} - e^{-1.75}}{e^{1.75} + e^{-1.75}}$$

Let's calculate this value:

$$Y_{\text{tanh}} \approx \frac{5.75 - 0.17377}{5.75 + 0.17377} \approx \frac{5.57623}{5.92377} \approx 0.941$$

So, the output Y_{tanh} using the tanh activation function is approximately **0.941**.

Summary of Results:

- Sigmoid Activation Output Y_{sigmoid} : Approximately **0.851**
- Tanh Activation Output Y_{tanh} : Approximately **0.941**

These are the outputs of the neuron using the sigmoid and tanh activation functions given the specific inputs, weights, and bias.

Numerical 2

To calculate the output of a neuron Y in a neural network using both the **sigmoid and tanh activation functions**.

Given:

- Inputs: $x_1 = 1.0, x_2 = -2.0, x_3 = 3.0$
- Weights: $w_1 = 0.2, w_2 = -0.5, w_3 = 0.7$
- Bias: $b = -0.4$

Step 1: Compute the Weighted Sum z

The weighted sum z is calculated as follows:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

Substituting the given values:

$$z = (0.2 \times 1.0) + (-0.5 \times -2.0) + (0.7 \times 3.0) - 0.4$$

$$z = 0.2 + 1.0 + 2.1 - 0.4 = 2.9$$

So, the weighted sum z is **2.9**.

Step 2: Apply the Sigmoid Activation Function

The sigmoid activation function is given by:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Substituting $z = 2.9$:

$$Y_{\text{sigmoid}} = \frac{1}{1 + e^{-2.9}}$$

To calculate $e^{-2.9}$:

$$e^{-2.9} \approx 0.055$$

Now calculate Y_{sigmoid} :

$$Y_{\text{sigmoid}} = \frac{1}{1 + 0.055} \approx \frac{1}{1.055} \approx 0.948$$

So, the output Y_{sigmoid} using the sigmoid activation function is approximately **0.948**.

Step 3: Apply the Tanh Activation Function

The tanh activation function is given by:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Substituting $z = 2.9$:

$$Y_{\text{tanh}} = \frac{e^{2.9} - e^{-2.9}}{e^{2.9} + e^{-2.9}}$$

First, calculate $e^{2.9}$ and $e^{-2.9}$:

$$e^{2.9} \approx 18.174 \quad \text{and} \quad e^{-2.9} \approx 0.055$$

Now calculate Y_{tanh} :

$$Y_{\text{tanh}} = \frac{18.174 - 0.055}{18.174 + 0.055} \approx \frac{18.119}{18.229} \approx 0.994$$

So, the output Y_{tanh} using the tanh activation function is approximately **0.994**.

Result: Using, Sigmoid Activation Output Y_{sigmoid} : Approximately 0.948 & using, Tanh Activation Output Y_{tanh} : Approximately 0.994.

Rectified Linear Unit (ReLU)

The Rectified Linear Unit (ReLU) is one of the most widely used activation functions in artificial neural networks (ANNs), particularly in deep learning models. It has become popular because of its simplicity and effectiveness in overcoming some of the limitations of earlier activation functions like the sigmoid and tanh.

It is defined by the following mathematical function:

$$\text{ReLU}(z) = \max(0, z)$$

Where:

- z is the input to the neuron, which is typically the weighted sum of inputs plus a bias term.
- The ReLU function outputs z if z is positive; otherwise, it outputs 0.

Key Properties of ReLU:

1. **Non-linearity:**
 - ReLU is a non-linear function, which means that it can model complex relationships in data. Despite its simplicity, ReLU allows neural networks to capture and learn complex patterns.
2. **Sparsity:**
 - Because ReLU outputs zero for any negative input, it tends to produce sparse activations (many neurons output zero). This sparsity can lead to more efficient computations and may also help in reducing the risk of overfitting.
3. **Computational Efficiency:**
 - ReLU is computationally efficient because it requires only a simple comparison operation (checking if the input is greater than zero) and does not involve any expensive exponential functions, unlike sigmoid or tanh.
4. **Avoiding the Vanishing Gradient Problem:**
 - ReLU helps to mitigate the vanishing gradient problem that is commonly encountered with sigmoid or tanh functions. In deep networks, gradients of these functions can become very small, slowing down learning. ReLU maintains a gradient of 1 for positive inputs, which helps keep gradients from becoming too small during backpropagation.

Variants of ReLU:

Several variants of the ReLU function have been proposed to address some of its drawbacks, such as:

Several variants of the ReLU function have been proposed to address some of its drawbacks, such as:

1. Leaky ReLU:

- Leaky ReLU allows a small, non-zero gradient when the input is negative, which can help with the "dying ReLU" problem (where neurons stop learning because they get stuck in the zero gradient zone).
- Defined as:

$$\text{Leaky ReLU}(z) = \max(0.01z, z)$$

2. Parametric ReLU (PReLU):

- Similar to Leaky ReLU, but the slope for negative inputs is a learnable parameter rather than a fixed value.
- Defined as:

$$\text{PReLU}(z) = \max(\alpha z, z)$$

where α is a learned parameter.

3. Exponential Linear Unit (ELU):

- ELU adds a smoothing effect to the negative side, helping the network to converge faster and with better accuracy.
- Defined as:

$$\text{ELU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha(e^z - 1) & \text{if } z \leq 0 \end{cases}$$

where α is a parameter that controls the saturation value for negative inputs.

ReLU in Neural Networks:

In a neural network, the ReLU function is applied to the output of each neuron after computing the weighted sum of inputs. This allows the network to introduce non-linearities, enabling it to model more complex functions.

For example, consider a neuron in a neural network with inputs $x_1, x_2, \dots, x_n$ corresponding weights $w_1, w_2, \dots, w_n$ and a bias b . The output of this neuron with ReLU activation is:

1. Compute the weighted sum z :

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

2. Apply the ReLU function:

$$Y = \text{ReLU}(z) = \max(0, z)$$

Advantages of ReLU:

- **Simple to implement** and computationally efficient.
- **Reduces the likelihood of the vanishing gradient problem**, making it suitable for deep networks.
- **Encourages sparse activations**, which can improve network efficiency and reduce overfitting.

Drawbacks of ReLU:

- **Dying ReLU problem:** Neurons can "die" during training if they consistently output zero (due to negative inputs), effectively becoming inactive. This can stop these neurons from learning.
- **Unbounded Output:** The output is not limited, which can sometimes lead to exploding activations in some network layers.

Numerical on ReLU

To calculate the output of a neuron Y in a neural network using **Rectified Linear Unit (ReLU)**

Steps to Calculate the Output of a Neuron Using ReLU:

Suppose we have the following values:

- Inputs: $x_1 = 1.0$, $x_2 = -2.0$, $x_3 = 3.0$
- Weights: $w_1 = 0.2$, $w_2 = -0.5$, $w_3 = 0.7$
- Bias: $b = -0.4$

Step 1: Compute the Weighted Sum z

The weighted sum z is calculated using the formula:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

Substituting the given values:

$$z = (0.2 \times 1.0) + (-0.5 \times -2.0) + (0.7 \times 3.0) - 0.4$$

$$z = 0.2 + 1.0 + 2.1 - 0.4 = 2.9$$

So, the weighted sum z is **2.9**.

Step 2: Apply the ReLU Activation Function

The ReLU activation function is given by:

$$\text{ReLU}(z) = \max(0, z)$$

Substituting $z = 2.9$:

$$Y_{\text{ReLU}} = \max(0, 2.9) = 2.9$$

To calculate the output of a neuron Y in a neural network using Leaky Rectified Linear Unit (Leaky ReLU)

Leaky ReLU Activation Function:

The Leaky ReLU activation function is defined as:

$$\text{Leaky ReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}$$

Where:

- z is the input to the activation function (i.e., the weighted sum of inputs plus a bias).
- α is a small positive constant (typically around 0.01) that determines the slope for negative inputs. This ensures that the function has a small, non-zero gradient when z is negative, which helps to avoid the "dying ReLU" problem.

Example Calculation:

Let's assume the following values for inputs, weights, and bias:

- Inputs: $x_1 = 1.5, x_2 = -2.0, x_3 = 1.0$
- Weights: $w_1 = 0.3, w_2 = -0.6, w_3 = 0.9$
- Bias: $b = 0.2$
- Leaky ReLU slope $\alpha = 0.01$

Step 1: Compute the Weighted Sum z

The weighted sum z is calculated as:

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$$

Substituting the given values:

$$z = (0.3 \times 1.5) + (-0.6 \times -2.0) + (0.9 \times 1.0) + 0.2$$

$$z = 0.45 + 1.2 + 0.9 + 0.2 = 2.75$$

So, the weighted sum z is **2.75**.

Step 2: Apply the Leaky ReLU Activation Function

Now, we apply the Leaky ReLU function:

$$Y_{\text{Leaky ReLU}} = \begin{cases} 2.75 & \text{if } 2.75 > 0 \\ 0.01 \times 2.75 & \text{if } 2.75 \leq 0 \end{cases}$$

Since $z = 2.75$ is greater than 0:

$$Y_{\text{Leaky ReLU}} = 2.75$$

Example with a Negative z Value:

For completeness, let's consider an example where z is negative.

Suppose we have:

- Inputs: $x_1 = -1.5, x_2 = 2.0, x_3 = -1.0$
- Weights: $w_1 = 0.3, w_2 = -0.6, w_3 = 0.9$
- Bias: $b = -0.2$

Now calculate z :

$$z = (0.3 \times -1.5) + (-0.6 \times 2.0) + (0.9 \times -1.0) - 0.2$$
$$z = -0.45 - 1.2 - 0.9 - 0.2 = -2.75$$

Now apply the Leaky ReLU function:

$$Y_{\text{Leaky ReLU}} = \begin{cases} -2.75 & \text{if } -2.75 > 0 \\ 0.01 \times -2.75 & \text{if } -2.75 \leq 0 \end{cases}$$

Since $z = -2.75$ is less than 0:

$$Y_{\text{Leaky ReLU}} = 0.01 \times -2.75 = -0.0275$$

Result:

- For $z = 2.75$, the Leaky ReLU output is **2.75**.
- For $z = -2.75$, the Leaky ReLU output is **-0.0275**.

Leaky ReLU function allows a small, non-zero gradient for negative values, helping the neuron continue to learn even when the input z is negative.

Softmax activation function

The softmax activation function is commonly used in the **output layer of a neural network**, especially in **multi-class classification problems**. It converts raw output scores (often called logits) into probabilities, with the sum of all probabilities equal to 1.

Softmax Function Definition:

For a neuron Y_i in a neural network, the softmax function is defined as:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Where:

- z_i is the weighted sum of inputs plus bias for the i -th neuron.
- n is the total number of neurons in the output layer.
- The numerator e^{z_i} is the exponential of the i -th neuron's weighted sum.
- The denominator $\sum_{j=1}^n e^{z_j}$ is the sum of the exponentials of all neurons' weighted sums, ensuring that the output values are probabilities that sum to 1.

Numerical

Suppose you have a neural network with 3 output neurons. Let's calculate the softmax output for each neuron.

Given:

- Weighted sums (logits) for the neurons:
 - $z_1 = 2.0$
 - $z_2 = 1.0$
 - $z_3 = 0.1$

Step 1: Calculate the Exponential of Each Logit:

First, calculate the exponentials of each z_i :

$$e^{z_1} = e^{2.0} \approx 7.389$$

$$e^{z_2} = e^{1.0} \approx 2.718$$

$$e^{z_3} = e^{0.1} \approx 1.105$$

Step 2: Calculate the Sum of All Exponentials:

Next, sum all the exponentials:

$$\sum_{j=1}^3 e^{z_j} = 7.389 + 2.718 + 1.105 \approx 11.212$$

Step 3: Compute the Softmax Output for Each Neuron:

Now, compute the softmax output for each neuron:

1. For Neuron 1:

$$\text{Softmax}(z_1) = \frac{e^{z_1}}{\sum_{j=1}^3 e^{z_j}} = \frac{7.389}{11.212} \approx 0.659$$

2. For Neuron 2:

$$\text{Softmax}(z_2) = \frac{e^{z_2}}{\sum_{j=1}^3 e^{z_j}} = \frac{2.718}{11.212} \approx 0.242$$

3. For Neuron 3:

$$\text{Softmax}(z_3) = \frac{e^{z_3}}{\sum_{j=1}^3 e^{z_j}} = \frac{1.105}{11.212} \approx 0.099$$

Results:

- Softmax output for Neuron 1: Approximately 0.659 (65.9%)
- Softmax output for Neuron 2: Approximately 0.242 (24.2%)
- Softmax output for Neuron 3: Approximately 0.099 (9.9%)

These softmax outputs can be interpreted as the probabilities that the input belongs to the class represented by each neuron. The neuron with the **highest softmax output** corresponds to the predicted class.

In this case, Neuron 1 has the highest softmax output, so the network would predict the class associated with Neuron 1.